

# SOCIALSPHEAR

## Engineering Roadmap

12 Milestones | 9-Month Plan | June 2026 | Confidential

## Overview

This roadmap covers 12 engineering milestones from initial project setup to the launch of P2 features. Each milestone has a clear goal, owner, and deliverable list. The 9-month timeline targets a public launch at Month 5 and full P1+P2 feature parity by Month 9.

| Milestone | Name                       | Month       | Track          | Status      |
|-----------|----------------------------|-------------|----------------|-------------|
| M1        | Foundation and Infra Setup | 1, Week 1–2 | Infrastructure | Not Started |
| M2        | Social Account OAuth       | 1, Week 3–4 | Feature        | Not Started |
| M3        | Content Composer           | 2, Week 1–2 | Feature        | Not Started |
| M4        | Scheduling Engine          | 2, Week 3–4 | Feature        | Not Started |
| M5        | Content Calendar           | 3, Week 1   | Feature        | Not Started |
| M6        | AI Caption and Hashtags    | 3, Week 2   | AI             | Not Started |
| M7        | Analytics Dashboard        | 3, Week 3–4 | Feature        | Not Started |
| M8        | Team and Approval Workflow | 4, Week 1   | Feature        | Not Started |
| M9        | Billing and Closed Beta    | 4, Week 2–3 | Launch         | Not Started |
| M10       | Public Launch              | 5, Week 1   | Launch         | Not Started |
| M11       | P1 Feature Sprint          | 5–6         | Feature + AI   | Not Started |
| M12       | P2 Feature Sprint          | 7–9         | Growth         | Not Started |

## Phase 1 — Build (Months 1–4)

### M1

Month 1, Week 1–2

Owner: Dipesh +  
Full-Stack  
Engineer 1

#### Foundation and Infrastructure Setup

Goal: Zero-to-running local dev environment and cloud infrastructure

- Initialise Turborepo monorepo with pnpm workspaces
- Scaffold Next.js 15 app with App Router, TypeScript, Tailwind CSS v4, Shadcn/UI
- Scaffold NestJS app with TypeScript, tRPC adapter, Swagger
- Create packages/types, packages/db, packages/ui in monorepo
- Set up Prisma schema v1 — User, Workspace, TeamMember tables
- Provision Supabase project — connect Prisma, run initial migration
- Set up Upstash Redis — connect to NestJS
- Integrate Clerk — sign-in, sign-up pages, ClerkGuard NestJS middleware
- Set up GitHub repository with branch protection rules
- Configure GitHub Actions — lint, typecheck, test, build pipeline
- Deploy Next.js to Vercel (preview and production environments)
- Deploy NestJS to Railway (staging and production environments)
- Set up Sentry for frontend and backend error tracking
- Set up PostHog for product analytics
- Write project README and local setup documentation

### M2

Month 1, Week 3–4

Owner: Full-Stack  
Engineer 1 +  
Backend Engineer

#### Social Account OAuth Connections

Goal: Users can connect and manage their social media accounts

- Create SocialAccount Prisma model — encrypted token fields
- Implement AES-256-GCM token encryption and decryption service
- Build OAuth 2.0 flow for: Instagram, Facebook, Twitter/X, LinkedIn, TikTok
- Build OAuth 2.0 flow for: Pinterest, YouTube, Google Business, Threads, BlueSky
- Create social-accounts NestJS module with connect/disconnect/refresh endpoints
- tRPC router: socialAccounts.connect, disconnect, list, getHealth
- Build account health checker — detect expired and expiring tokens
- Implement Temporal TokenRefreshWorkflow — auto-refresh 24h before expiry
- Frontend: Connected Accounts settings page with add/remove/status UI
- Frontend: OAuth redirect handler and success/error states
- Handle platform API rate limits — per-platform request throttling
- Write integration tests for each platform OAuth flow (mocked with MSW)

### M3

Month 2, Week 1–

#### Content Composer

Goal: Users can create, preview, and draft multi-platform posts with media

2

Owner: Frontend  
Engineer + Full-  
Stack Engineer 1

- Build rich text editor component (Tiptap or custom contenteditable)
- Real-time character counter per selected platform with limit enforcement
- Platform selector — choose 1 or many platforms per post
- Per-platform content overrides — different caption per platform if needed
- Media upload: drag-and-drop, file picker — images and videos
- R2 presigned upload URL generation via NestJS media module
- Cloudinary integration — auto-transform uploaded media per platform format
- Multi-image carousel support for Instagram and LinkedIn
- Platform preview panel — live preview of how post looks on each platform
- Hashtag input with typeahead suggestions (Typesense-backed)
- Draft auto-save — Zustand draft store, debounced persistence to DB every 30s
- Posts tRPC router: create, update, saveDraft, deleteDraft
- Frontend: Composer page layout with editor, preview, and AI sidebar panels
- Playwright E2E: create post, upload image, preview, save draft

M4

Month 2, Week 3–  
4

Owner: Backend  
Engineer + Full-  
Stack Engineer 2

## Scheduling Engine — BullMQ + Temporal

Goal: Posts publish reliably at the scheduled time with retry and alerting

- Set up BullMQ with Upstash Redis — all 6 queues defined
- Build scheduler NestJS module — PostSchedulerService
- Build publisher NestJS module — separate workers per social platform
- Implement platform publisher adapters: Instagram Graph API, Facebook, Twitter/X v2
- Implement platform publisher adapters: LinkedIn, TikTok, Pinterest, YouTube, others
- Post state machine — DRAFT to SCHEDULED to QUEUED to PUBLISHING to PUBLISHED or FAILED
- BullMQ retry logic — 3 retries with exponential backoff per queue spec
- Temporal ScheduledPostWorkflow — wait, dispatch, monitor, analytics trigger
- Temporal RecurringPostWorkflow — clone and reschedule on completion
- Failed post alert — Socket.io event and email notification via Resend
- PostJob table population — one row per platform per post, status tracking
- BullMQ dashboard (Bull Board) — internal admin view of queue health
- Stress test: schedule 1,000 posts simultaneously, verify under 1% failure rate
- Playwright E2E: schedule post, verify PostJob status, mock delivery success

## M5

Month 3, Week 1

Owner: Frontend Engineer + Full-Stack Engineer 1

### Content Calendar

Goal: Users see all scheduled posts in a visual calendar and can drag to reschedule

- Month view calendar — all scheduled posts per day, colour-coded by platform
- Week view calendar — hourly slots with post cards
- Day view calendar — full post details per slot
- Drag-and-drop rescheduling — update scheduledAt on drop with Zustand optimistic update
- Platform filter — show and hide specific platforms
- Status filter — scheduled, published, failed, draft
- Post quick-preview popover on hover — content, platform, scheduled time
- Post edit — click post to open in composer
- Bulk select — select multiple posts, bulk reschedule or delete
- Calendar export — PDF via browser print, CSV via tRPC endpoint
- Virtual rendering for calendars with 500+ posts (TanStack Virtual)
- Responsive: full calendar functionality on mobile with touch drag-and-drop

## M6

Month 3, Week 2

Owner: AI/ML Engineer + Frontend Engineer

### AI Caption Generator and Hashtag Engine

Goal: Users generate platform-optimised captions and hashtags with one click

- NestJS AI module — CaptionGeneratorService, HashtagService, BrandVoiceService
- Integrate Vercel AI SDK with Anthropic Claude API (claude-sonnet-4-6)
- Caption generation tRPC procedure — topic, tone, platform, language inputs
- Streaming response — stream caption tokens to frontend via tRPC subscription
- Platform-specific formatting — auto-trim to character limit, adjust tone
- Tone selector — professional, casual, witty, inspirational, educational
- Language support — 20 languages via Claude API
- Hashtag generation — suggest 5–30 relevant hashtags per topic
- Regenerate flow — Give me 3 alternatives — stream 3 options simultaneously
- Brand voice training UI — upload 5–20 example posts in AI Studio
- Brand voice embedding pipeline — pgvector storage, cosine similarity retrieval
- AI credit system — deduct credits per generation, check plan limits
- Credit usage display — current usage vs plan allowance in header
- Frontend: AI assist sidebar in composer — caption, hashtags, tone, language

## M7

Month 3, Week 3–4

Owner: Backend Engineer + Frontend Engineer

### Analytics Dashboard

Goal: Users see post and account performance metrics with date range filtering

- Provision ClickHouse Cloud instance — post\_metrics and account\_snapshots tables
- Analytics ingestion worker — BullMQ analytics:ingest queue fetches metrics 1h after publish
- Platform analytics adapters — pull engagement data from each social platform API

- ClickHouse query service — NestJS AnalyticsQueryService
- tRPC analytics router: `getPostMetrics`, `getAccountMetrics`, `getTopPosts`, `exportCSV`
- Dashboard overview: total reach, impressions, engagement rate (last 30 days)
- Top posts ranked by engagement rate — filterable by platform and date range
- Follower growth chart — line chart per platform (Recharts)
- Engagement rate over time — area chart with platform overlay
- Platform comparison — bar chart comparing reach across platforms
- Post-level breakdown — click any post to see full metrics history
- Date range filter: 7d, 30d, 90d, custom with date picker
- CSV export — all metrics for selected date range as downloadable file
- Mobile-responsive charts with touch-friendly interactions

## M8

Month 4, Week 1

Owner: Full-Stack Engineer 1 + Full-Stack Engineer 2

### Team and Approval Workflow

Goal: Teams can collaborate with roles, invites, and a content approval flow

- TeamMember Prisma model — `workspace_id`, `user_id`, `role`, `invite_status`
- tRPC workspace router: `invite`, `acceptInvite`, `updateRole`, `removeMember`, `listMembers`
- Role-based access control middleware — Owner, Admin, Editor, Viewer permissions
- Email invite flow — Resend sends invite link, user clicks to join workspace
- Approval workflow: Editor submits post, Admin receives notification, approve or reject
- Approval status on PostJob — `none`, `pending`, `approved`, `rejected`
- Scheduled posts blocked from BullMQ queue until `approval_status` is approved
- Rejection reason — Admin adds comment on rejection, Editor notified
- Activity log — append-only log of all post and member actions in workspace
- Frontend: Team settings page — member list, invite modal, role dropdowns
- Frontend: Approval queue view — pending posts for Admin review
- Frontend: Notification bell — real-time Socket.io count of pending approvals
- Frontend: Activity log page — filterable by user, action, date

## Phase 2 — Launch (Months 4–5)

### M9

Month 4, Week 2–3

Owner: Full-Stack Engineer 2 + Dipesh (GTM)

#### Stripe Billing and Closed Beta

Goal: Platform is billable and 50 beta users are actively using it

- Create Stripe products and prices for all 5 plans (Free, Creator, Pro, Agency, Enterprise)
- NestJS billing module — CheckoutService, SubscriptionService, WebhookService
- Stripe Checkout integration — upgrade flow from within dashboard
- Stripe webhook handler — subscription created/updated/cancelled updates workspace.plan
- Plan enforcement middleware — check plan limits before AI generation or channel add
- Billing portal — customers manage their subscription via Stripe Customer Portal
- Usage-based credit tracking — AI credits consumed vs plan allowance in Redis
- Frontend: Pricing page at /pricing — all plans with feature comparison table
- Frontend: Upgrade prompt modals — shown when user hits plan limit
- Frontend: Billing settings page — current plan, usage, upgrade/downgrade button
- Recruit 50 beta users from Twitter/X, LinkedIn, and creator communities
- Beta feedback form in-app — PostHog survey triggered after 3rd post scheduled
- Weekly beta user feedback calls — Dipesh leads, product issues logged in GitHub

### M10

Month 5, Week 1

Owner: Dipesh (GTM) + All Engineers

#### Public Launch

Goal: SocialSphear is publicly available with paying customers and press coverage

- ProductHunt launch — prepare assets, schedule launch, coordinate upvote campaign
- Launch blog post — Why we built SocialSphear on socialsphear.com/blog
- Twitter/X launch thread — Dipesh personal account, behind-the-scenes build story
- YouTube demo video — 5-minute walkthrough of scheduling flow and AI features
- Landing page — hero, features, pricing, testimonials, FAQ
- SEO foundation — meta tags, Open Graph, sitemap.xml, robots.txt, Core Web Vitals green
- Affiliate programme setup — 30% recurring commission via Rewardful
- Founding Member offer — first 100 customers get 50% off for life
- Customer success playbook — onboarding email sequence (5 emails over 14 days)
- Help documentation — socialsphear.com/docs with guides for all core flows
- Status page — status.socialsphear.com (Better Uptime)
- Support inbox — Intercom live chat on dashboard, support@socialsphear.com
- Agency cold outreach — 200 LinkedIn DMs to social media agency founders
- Press outreach — TechCrunch, IndieHackers, Hacker News Show HN

## Phase 3 — Growth (Months 5–9)

### M11

Month 5–6

Owner: All  
Engineers

#### P1 Feature Sprint

Goal: 8 high-demand features ship based on beta user feedback

- AI image generation — fal.ai Flux integration, 512px and 1024px options
- Image generation UI — prompt input, style options, generated image gallery in composer
- Link-in-bio page builder — custom landing page at [socialsphear.com/u/username](https://socialsphear.com/u/username)
- Instagram grid preview — virtual grid showing how scheduled posts change the feed
- Social inbox — unified comment and DM feed from all connected platforms
- Social inbox reply interface — reply to comments inline with platform preview
- White-label PDF reports — agency plan, custom logo and brand colours, client name
- Report builder — select metrics, date range, platforms, generate PDF via Puppeteer
- RSS auto-posting — connect blog RSS, new posts auto-drafted for scheduling
- Bulk CSV scheduling — upload CSV with content, dates, platforms, media URLs
- Browser extension — Chrome and Firefox, clip content to drafts queue from any webpage
- Canva integration — OAuth connect, import design directly to composer

### M12

Month 7–9

Owner: All  
Engineers +  
DevOps

#### P2 Feature Sprint

Goal: Advanced features unlock agency and enterprise market segments

- Social listening — keyword and brand mention monitoring across platforms
- Listening alert engine — email and Slack notifications for negative sentiment spikes
- Competitor tracking — track up to 5 competitor accounts, follower growth, posting frequency
- AI content planner — LangChain agent generates full month content calendar
- Content plan UI — AI-suggested posts loaded into calendar, editable before scheduling
- Video caption generation — auto-transcribe video uploads using Whisper API
- TikTok native auto-publish — full Content Posting API (no notification workaround)
- Instagram Reels native auto-publish — using Reels Publishing API
- Zapier integration — publish SocialSphear Zapier app to marketplace
- Make integration — SocialSphear module with trigger and action support
- Public REST API — documented at [api.socialsphear.com](https://api.socialsphear.com), OAuth 2.0, rate limited
- API key management UI — create, revoke, view usage for developer API keys
- Slack notification integration — post publish confirmations and approval requests
- SOC 2 Type I preparation — engage auditor, implement controls, gather evidence

## Appendix: Engineering Standards

### Definition of Done

- Feature has passing unit tests (Vitest) with over 80% coverage of new code
- E2E test written in Playwright covering the happy path
- TypeScript strict mode — no 'any' types, no type assertions without comment
- Reviewed and approved by at least one other engineer via GitHub PR
- Deployed to staging and smoke tested manually
- PostHog event tracking added for key user actions in the feature
- Documentation updated in /docs if feature changes a user-facing flow

### Engineering Principles

- Ship working software over perfect software — get features to beta users fast
- TypeScript everywhere — if it does not have types, it does not ship
- Test what matters — E2E for critical paths (scheduling, billing), unit for business logic
- Database migrations are forward-only — no destructive migrations without data backfill
- Feature flags via PostHog — all major features gated behind a flag before full rollout
- Error budgets — if post delivery success rate drops below 99.5%, halt all non-critical work

### Team Hiring Timeline

| Role                       | Hire By | Stack                                       | Salary Range (USD) |
|----------------------------|---------|---------------------------------------------|--------------------|
| Product Designer (UI/UX)   | Month 1 | Figma, design systems, user research        | \$70k–\$90k        |
| Full-Stack Engineer 1      | Month 1 | Next.js, NestJS, TypeScript, Prisma         | \$90k–\$120k       |
| Backend Engineer           | Month 2 | NestJS, BullMQ, Temporal, ClickHouse        | \$90k–\$120k       |
| Frontend Engineer          | Month 2 | Next.js, Tailwind, Shadcn, Recharts         | \$80k–\$110k       |
| Full-Stack Engineer 2      | Month 3 | Next.js, NestJS, TypeScript                 | \$90k–\$120k       |
| AI/ML Engineer             | Month 3 | LangChain.js, Vercel AI SDK, vector DBs     | \$110k–\$140k      |
| DevOps Engineer (contract) | Month 2 | Railway, Cloudflare, GitHub Actions, Docker | \$50/hr            |